🧠 Complete Project Breakdown (Clear & Deep Version)

🎯 1. What Problem Are You Solving?

Your system solves a **real-world data problem** called:

👉 **Fuzzy Record Linkage** (also called Entity Resolution)

In simple words:

> "Given messy data, find records that refer to the same person."

⚠️ Why this is hard:

Real datasets are messy:

Real Case	Problem
`Houda Amer` vs `HOUDA AMER`	Case difference
`Houda Amer` vs `Houda Ammer`	Typo
`Amer Houda` vs `Houda Amer`	Word order
`Mustapha` vs `Mustafa`	Spelling variation

👉 A normal `equals()` comparison would FAIL.
👉 Your system intelligently says: "These are probably the same person."

🧩 2. The 3 Main Features (Use Cases)

🔍 1. Search (Recherche)

> Find matches of **one person inside a dataset**

Example:

Input: "Houda Amer"
Dataset: 658,000 names
Output: All similar names + scores

🔄 2. Compare (Comparaison)

> Match **two different datasets**

Example:

Dataset A: 658k names
Dataset B: 32k names

Goal: Find people appearing in BOTH datasets

♻️ 3. Deduplication

> Detect duplicates **inside one dataset**

Example:

"Houda Amer"
"Houda Ammer"
"Amer Houda"

→ All represent the same person → group them

🏗️ 3. Architecture Philosophy (VERY IMPORTANT)

Your system is based on:

👉 **Strategy Pattern**

💡 Idea:

Instead of hardcoding ONE algorithm, you make everything **replaceable**.

So the engine does NOT care:

- how you preprocess
- how you compare
- how you generate candidates

It just **executes what you plug in**.

🧠 Think of it like LEGO:

You build your system using blocks:

[Preprocessing] + [Candidate Generator] + [Comparator] + [Selector]

Change one block → behavior changes.

🔁 4. The Core Pipeline (Step-by-Step)

This is the HEART of your system.

RAW DATA (CSV)
↓
[1] PREPROCESSING
↓
[2] CANDIDATE GENERATION
↓
[3] SIMILARITY SCORING
↓
[4] SELECTION / FILTERING
↓
FINAL RESULTS

🧼 5. Stage 1 — Preprocessing (Cleaning Data)

Goal:

> Normalize text so comparisons are fair

Examples:

Before	After
`HOUDA AMER`	`houda amer`
`Houda, Amer!!`	`houda amer`
`amer houda`	`amer houda` (sorted tokens)

Your Preprocessors:

✔️ **Lowercase**
"HOUDA AMER" → "houda amer"

✔️ **Tokenization + Sorting**
"Houda Amer" → ["houda", "amer"] → sorted → "amer houda"

👉 Fixes order problems

✔️ **Remove special characters**
"Houda@Amer123" → "houda amer"

💡 Key Insight:

Preprocessing = **80% of accuracy**

Bad preprocessing → bad results even with good algorithms

⚡ 6. Stage 2 — Candidate Generation (Performance Trick)

❌ Problem:

If you compare EVERYTHING:

658k × 658k = 433 BILLION comparisons 😱

Impossible.

✅ Solution:

Only compare **likely matches**

🔹 Method 1: Token-Based Generator

Idea:

> People sharing words are likely similar

Example:

Query: "houda amer"

→ Find all records containing:
"houda" OR "amer"

👉 Reduces 658k → ~50k candidates

🔹 Method 2: Length-Based Generator

Idea:

Similar names have similar length

Example:

Length("Houda Amer") = 10

→ Accept names length between:
5 and 15 (±50%)

💡 Key Insight:

This stage is what makes your system **FAST**

📏 7. Stage 3 — Similarity Scoring

Now we compare strings mathematically.

🧠 1. Jaro-Winkler (BEST for names)

- Detects matching characters
- Handles swapped letters
- Bonus for same prefix

Example:

"Mustapha" vs "Mustafa" → 0.93 ✅

👉 Very good for human names

✏️ 2. Levenshtein Distance

Counts edits:

kitten → sitting
= 3 operations

Formula:

Similarity = 1 - (distance / max_length)

👉 More strict than Jaro-Winkler

⚖️ 3. Simple Comparator

exact match → 1
else → 0

👉 Fast but useless for real-world data

🎯 8. Stage 4 — Selection (Filtering Results)

After scoring, you filter results.

Options:

✔️ **Threshold**
Keep score ≥ 0.80

✔️ **Top N**
Keep best 100 matches

✔️ **All**
Keep everything (for analysis)

🧱 9. Data Model (How Data Flows)

CSV → CoupleDeString (raw)
↓
Nom (clean object)
↓
CoupleDeNom (pair to compare)
↓
NomAvecScore (final result)

⚙️ 10. The Engine (Brain of System)

Class:

MoteurDeRecherche

Main Methods:

🔍 **Search**
rechercher(query, dataset)

🔄 **Compare**
comparer(datasetA, datasetB)

♻️ **Deduplicate**
dedupliquer(dataset)

What it does internally:

1. Apply preprocessing
2. Generate candidates
3. Compute similarity
4. Filter results

🖥️ 11. GUI (User Interface)

Built using:
👉 Java Swing

Features:

- Select action (Search / Compare / Deduplicate)
- Choose datasets (.csv auto-detected)
- Choose algorithm (Jaro / Levenshtein)
- Toggle preprocessing
- Set threshold (slider)
- Limit results

⚡ Performance:

Uses:

SwingWorker (background thread)

👉 UI does NOT freeze

💻 12. CLI (Command Line)

Simple menu:

1. Search
2. Compare
3. Deduplicate
4. Configure

🧯 13. Critical Problems You Solved

❌ Memory Crash (OutOfMemoryError)

Cause:

Generating ALL pairs (quadratic explosion)

✅ Fix:

- Use HashSet to deduplicate
- Avoid dataset-to-dataset full comparison
- Remove nested loops

📦 Dependency Issue

You removed:
👉 Apache Commons Text

And implemented:

- Jaro-Winkler
- Levenshtein

👉 This is VERY impressive (good for jury 👏)

🔍 14. Real Execution Example

Search:

"Houda Amer"

Step-by-step:

1. Preprocess:

→ "houda amer"

2. Generate:

→ 50k candidates

3. Score:

→ compute similarity for each

4. Filter:

→ keep score ≥ 0.80

5. Output:

12 matches in ~4 seconds

🧠 15. Why Your Design is STRONG

✅ Modular
✅ Scalable
✅ Configurable
✅ Efficient
✅ No dependencies
✅ Real-world applicable

🚀 16. Advanced Improvements (If Jury Asks)

You can say:

- Add **Phonetic Matching** (Soundex)

- Add **N-grams blocking**
- Add **Machine Learning ranking**
- Export results (CSV/JSON)
- Build REST API (Spring Boot)

🧾 Final Simple Explanation (For Presentation)

If someone asks:

👉 *“Explain your project in 20 seconds”*

Say:

> “My project is a modular fuzzy matching engine that identifies similar person names across large datasets. It uses a configurable pipeline with preprocessing, candidate generation, similarity scoring, and filtering. Thanks to the strategy pattern, we can swap algorithms like Jaro-Winkler or Levenshtein easily while maintaining performance on datasets with hundreds of thousands of records.”